



西安电子科技大学
XIDIAN UNIVERSITY

计算机科学与技术学院
School of Computer Science and Technology

分布式计算

Distributed Computing

主讲人：李龙海

第六讲 Spark平台和基于RDD-DAG的计算模型



主要内容

- Spark平台简介
- 分布式弹性数据RDD
- 常用的RDD算子
- 用Spark实现简单的大数据处理任务



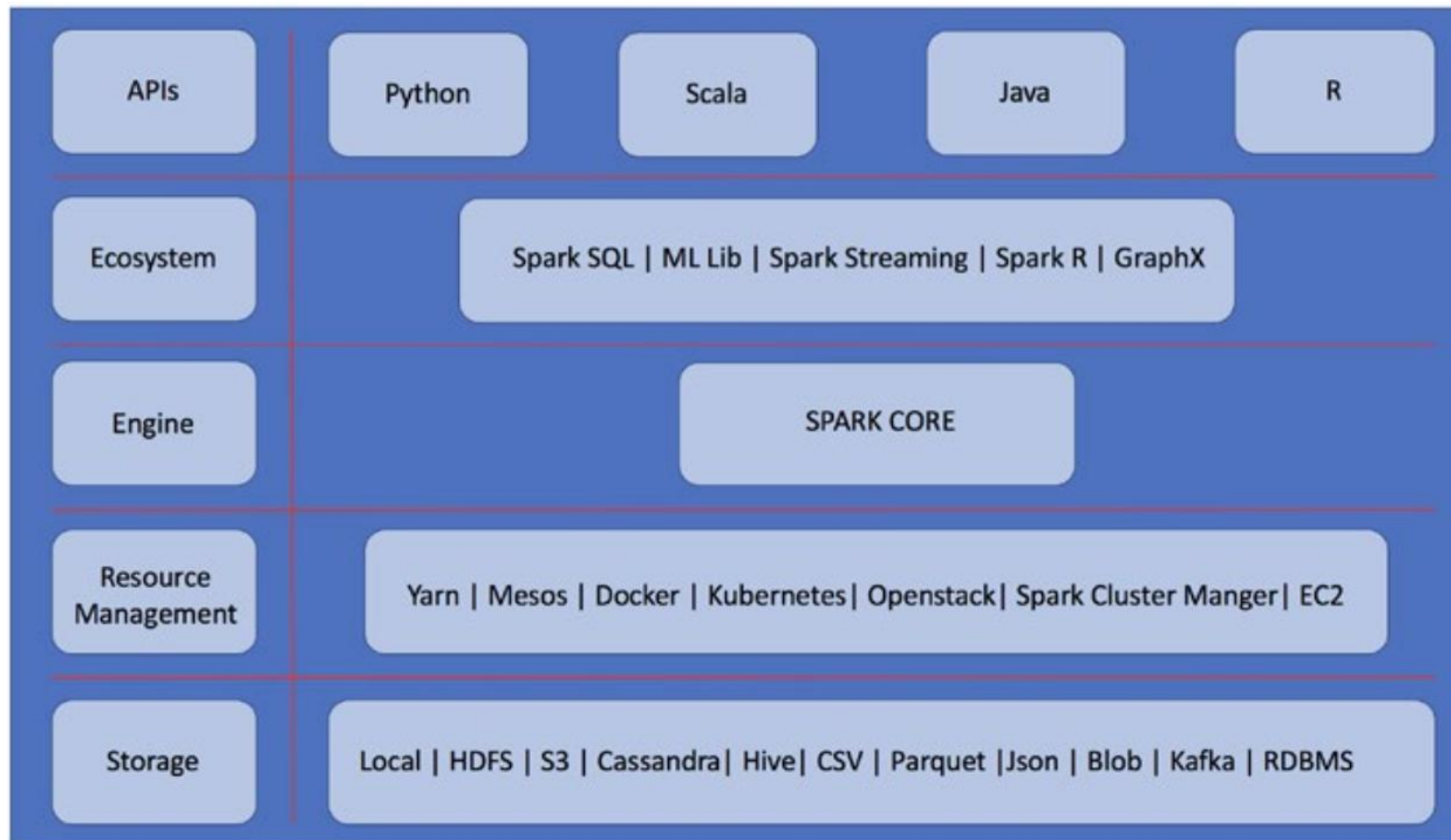
- 由加州大学伯克利设计，后成为 Apache 顶级开源项目
- 基于内存计算，运行速度快
- 支持大规模分布式并行处理
- 提供丰富的数据处理算子
- 支持 DAG（有向无环图）计算模型
- 适合迭代计算与交互式分析
- 编程接口简洁，开发效率高
- 可运行在 Hadoop YARN、Kubernetes 等集群平台
- 提供丰富生态系统（Spark SQL、流处理、机器学习、图计算）

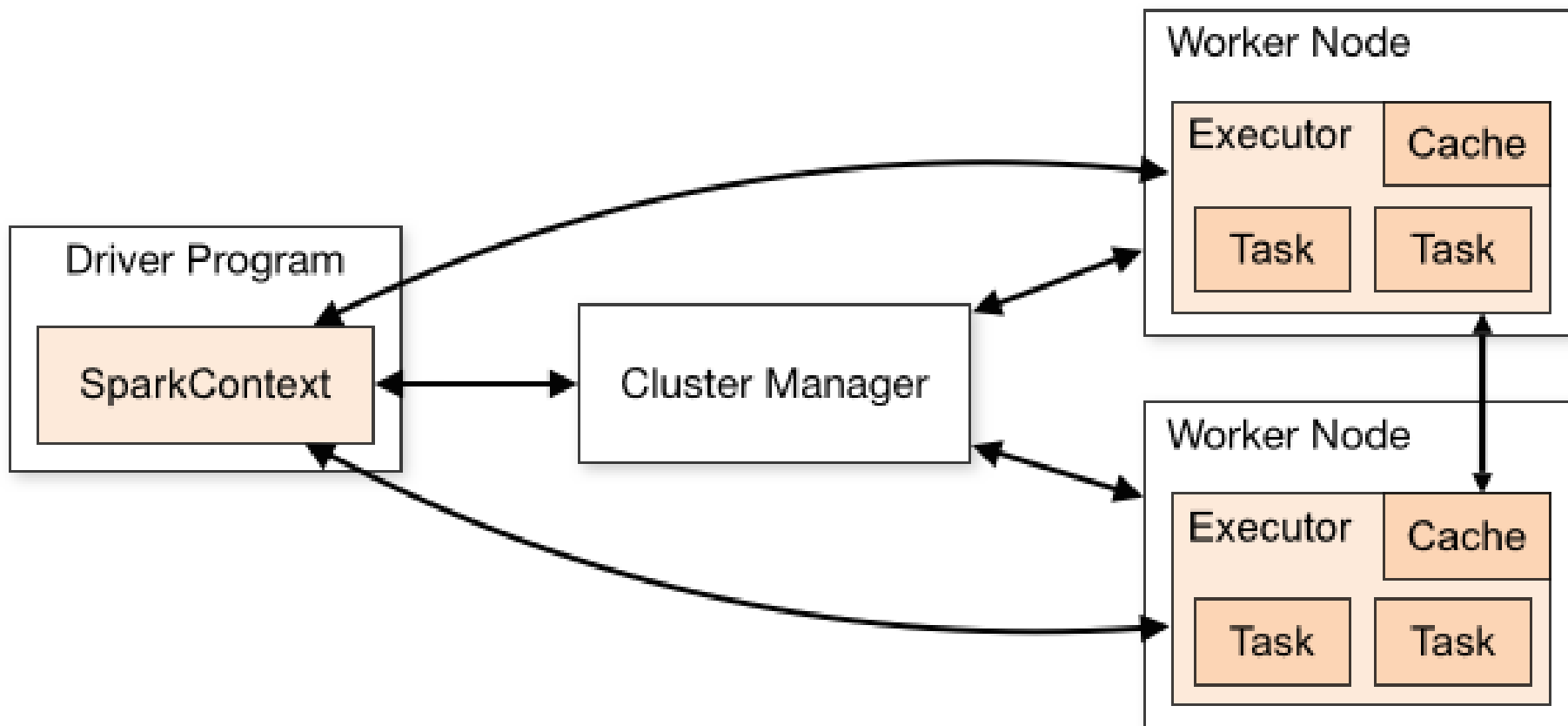


对比项	MapReduce	Spark
编程抽象	Map + Reduce	DAG (有向无环图)
数据模型	Key-Value数组	RDD / DataFrame
中间结果	磁盘	内存
迭代计算	慢	快
开发复杂度	高	低
执行效率	较低	较高
实时处理	不适合	支持
子任务优化	不支持	支持



应用程序







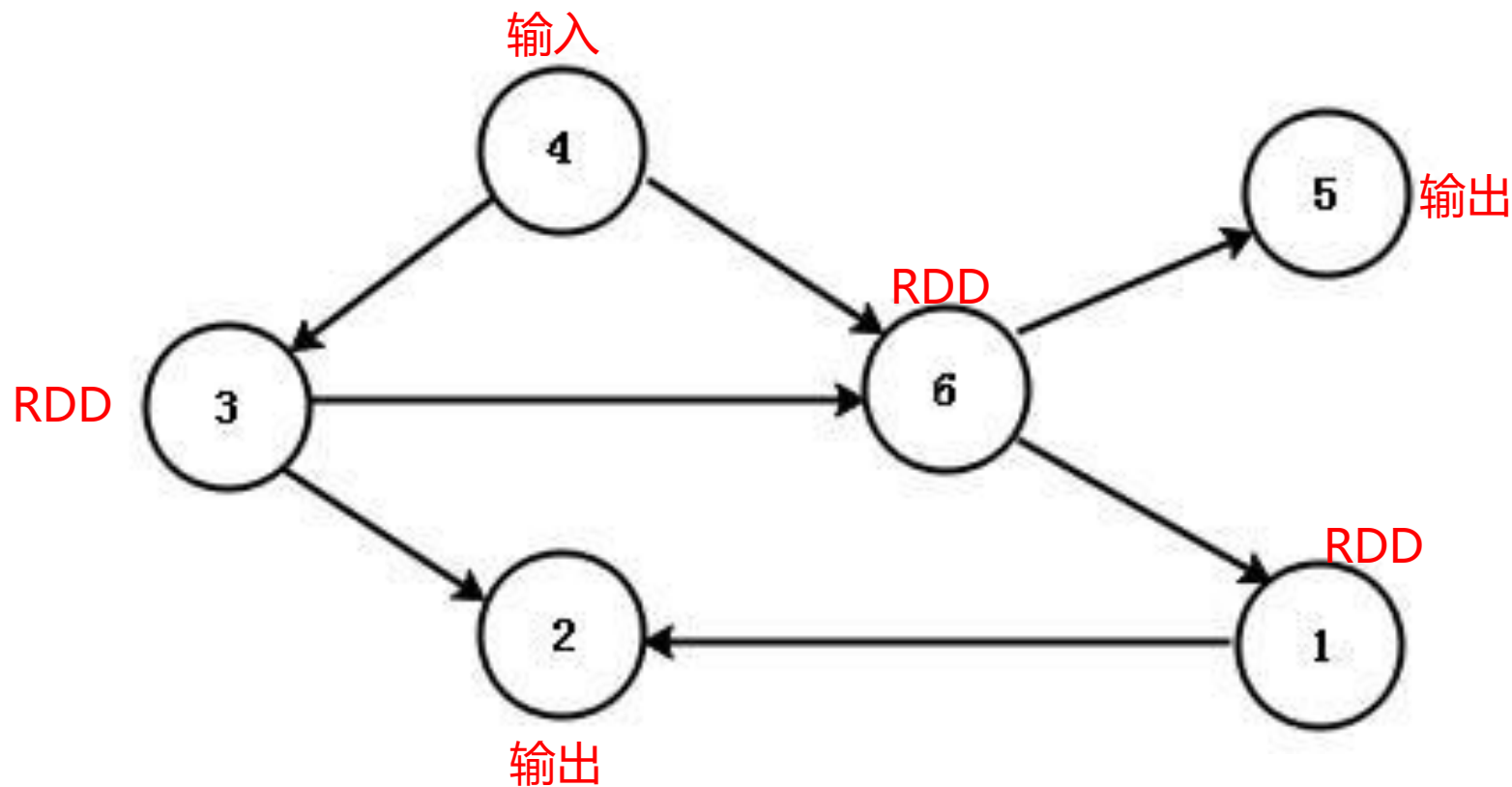
- RDD 全称为 Resilient Distributed Datasets，是 Spark 平台 最基本的数据抽象。可以将RDD看成是一个分布式存储的“巨大数组”。
- Spark应用程序只需关心如何由一个RDD转换为另一个RDD，不用关心RDD在底层是如何分区、如何分布到多个节点上、如何在内存中缓存、内存缓存丢失后如何重新生成。
- 利用Spark平台提供的RDD算子，实现RDD直接的转换。
- RDD 创建后不能修改，只能生成新的 RDD。
- 一个 RDD 由一个或者多个分区（Partitions）组成。对于 RDD 来说，每个分区会被一个计算子任务所处理，用户可以在创建 RDD 时指定其分区个数。如果没有指定，则平台会根据数据分布存储、CPU资源等情况自己决定。
- 关于RDD的一个分区的计算任务失败后，Spark平台会自动在其他计算节点上恢复该任务。（容错性）



- 一个具体的大数据处理任务可以表达为一系列RDD之间的转换。
- 一个分布式计算任务中涉及到的不同RDD 之间存在依赖关系，RDD 的每次转换都会生成一个新的依赖关系，这种RDD 之间的依赖关系就像流水线一样。RDD(s) 及其之间的依赖关系组成了 DAG(有向无环图)。
- 一个分布式计算任务可以表达为一个DAG。
- DAG 定义了这些 RDD(s) 之间的 Lineage(血统) 关系，通过血统关系，如果一个 RDD 的部分或者全部计算结果丢失了，也可以重新进行计算。
- Spark 可以根据 DAG对某些计算子任务进行合并。



将计算任务抽象为有向无环图

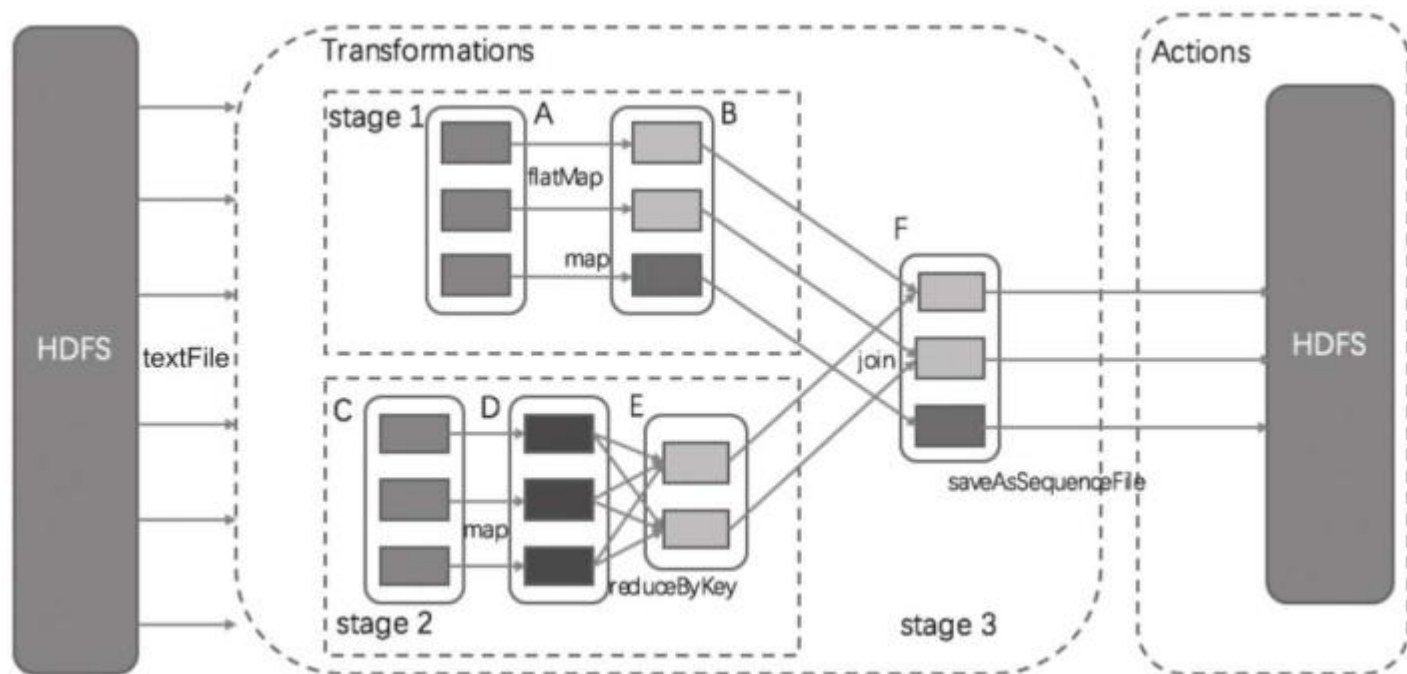
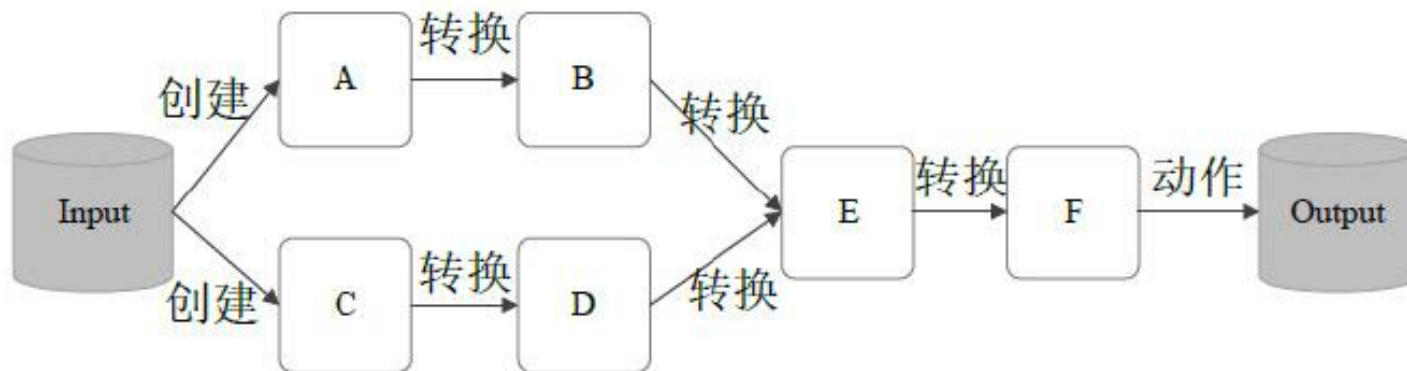


DAG中的节点表示三类对象：（1）输入文件；（2）输出文件；（3）**RDD**

DAG中有向边表示：**RDD**转换算子



将计算任务抽象为有向无环图





HDFS中的日志文件

lines

filter(_.startsWith("ERROR"))

errors

filter(_.contains("HDFS"))

HDFS errors

map(_.split('\t')(3))

time fields

```
lines = sc.textFile("hdfs://...")
errors = lines.filter(lambda x: x.startswith("ERROR"))
hdfserrors = errors.filter(lambda x: x.startswith("HDFS"))
...
errors.persist()
```



■ 创建RDD的算子：

- 基于外部存储系统上的文件创建：

```
lines = sc.textFile( "hdfs://localhost:9000/log.txt" )
```

- 基于驱动程序(Driver Program)的一个本地内存数组创建：

```
val list = List(3, 6, 9, 10, 12, 21)
```

```
lines = sc.parallelize(list)
```

■ Transformation算子：

- 将一个RDD转换成一个新的RDD。
- Transformation算子的动作是“惰性执行”的，即不是在定义时刻执行，只在必要时才执行。

■ Action算子：

- 在RDD上运行计算后将结果返回到驱动程序本地数组或者本地变量；或者在RDD上运行计算后将结果保存到外部存储系统上。
- Action算子定义的动作一般会立即执行，进而触发其它的惰性Transformation算子的执行。



Transformations	$map(f : T \Rightarrow U) : RDD[T] \Rightarrow RDD[U]$ $filter(f : T \Rightarrow Bool) : RDD[T] \Rightarrow RDD[T]$ $flatMap(f : T \Rightarrow Seq[U]) : RDD[T] \Rightarrow RDD[U]$ $sample(fraction : Float) : RDD[T] \Rightarrow RDD[T]$ (Deterministic sampling) $groupByKey() : RDD[(K, V)] \Rightarrow RDD[(K, Seq[V])]$ $reduceByKey(f : (V, V) \Rightarrow V) : RDD[(K, V)] \Rightarrow RDD[(K, V)]$ $union() : (RDD[T], RDD[T]) \Rightarrow RDD[T]$ $join() : (RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (V, W))]$ $cogroup() : (RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (Seq[V], Seq[W]))]$ $crossProduct() : (RDD[T], RDD[U]) \Rightarrow RDD[(T, U)]$ $mapValues(f : V \Rightarrow W) : RDD[(K, V)] \Rightarrow RDD[(K, W)]$ (Preserves partitioning) $sort(c : Comparator[K]) : RDD[(K, V)] \Rightarrow RDD[(K, V)]$ $partitionBy(p : Partitioner[K]) : RDD[(K, V)] \Rightarrow RDD[(K, V)]$
Actions	$count() : RDD[T] \Rightarrow Long$ $collect() : RDD[T] \Rightarrow Seq[T]$ $reduce(f : (T, T) \Rightarrow T) : RDD[T] \Rightarrow T$ $lookup(k : K) : RDD[(K, V)] \Rightarrow Seq[V]$ (On hash/range partitioned RDDs) $save(path : String) : \text{Outputs RDD to a storage system, e.g., HDFS}$

可参考教程：<https://github.com/heibaiying/BigData-Notes> 中

“弹性式数据集 RDD”

“RDD 常用算子详解”



```
from pyspark import SparkContext
```

```
sc = SparkContext("local", "Spark Example")
```

1. map算子（一对一映射）：将元素乘以2

```
rdd1 = sc.parallelize([1, 2, 3, 4]).map(lambda value: value * 2)
```

```
rdd1.collect().foreach(lambda x: print(x))
```

2. flatMap算子（一对多映射）：将每个字符串拆分成单个字符

```
rdd2 = sc.parallelize(["A,B", "C,D,E", "F,G"]).flatMap(lambda work: work.split(","))
```

```
rdd2.collect().foreach(lambda x: print(x))
```

3. filter算子：过滤掉包含"yo"的元素

```
rdd3 = sc.parallelize(["yo", "pai", "xc"]).filter(lambda work: "yo" not in work)
```

```
rdd3.collect().foreach(lambda x: print(x))
```

4. reduceByKey算子：对相同key的值进行累加

```
rdd4 = sc.parallelize([("勇哥", 100), ("勇哥", 98), ("小明", 97)]).reduceByKey(lambda a, b: a + b)
```

```
rdd4.collect().foreach(lambda x: print(x))
```

5. groupByKey算子：将相同key的元素分组

```
rdd5 = sc.parallelize([("勇哥", 100), ("小红", 98), ("小明", 97), ("小明", 77)])
```

```
rdd5.groupByKey().collect().foreach(lambda x: print(x))
```



```
from pyspark import SparkConf, SparkContext  
conf = SparkConf().setMaster("local").setAppName("wordcount")  
sc = SparkContext(conf=conf)  
textData = sc.textFile("./readme.txt")  
splitData = textData.flatMap(lambda line: line.split(" "))  
flagData = splitData.map(lambda word: (word, 1))  
countData = flagData.reduceByKey(lambda x, y: x + y)  
countData.saveAsTextFile("./result")
```



■ 题目1:

输入文件为学生成绩信息，包含了必修课与选修课成绩，格式如下：

班级1, 姓名1, 科目1, 必修, 成绩1
 （注：
 为换行符）

班级2, 姓名2, 科目1, 必修, 成绩2

班级1, 姓名1, 科目2, 选修, 成绩3

.....,,,

编写Spark程序，实现如下功能：

1. 计算每个学生必修课的平均成绩。
2. 按科目统计每个班的平均成绩。



本地仿真模式：

- `pip install pyspark`
- `spark-submit --master local[2] spark_wordcount.py`
- `python spark_wordcount.py` #和上面命令效果类似

在Hadoop平台上运行：

```
spark-submit --master yarn --deploy-mode cluster \  
spark_wordcount.py
```